Exploiting von Buffer-Overflows mit GDB

Grundlagenpraktikum zur IT-Sicherheit (WiSe 2023/2024)

Lehrstuhl für Netz- und Datensicherheit
Ruhr-Universität Bochum

Nico Steppeler
Matr.-Nr:1080 2221 3001
Nico.Steppeler@ruhr-uni-bochum.de

Sabri Öztürk
Matr.-Nr:1080 2122 7680
Sabri.Oeztuerk@ruhr-uni-bochum.de

27. April 2026

Zusammenfassung

In diesem Versuch wird gezeigt, wie ein Buffer-Overflow funktioniert und welche Auswirkungen ein solcher Angriff haben kann. Um die Funktionsweise nachvollziehen zu können, wird zu Beginn das Tool GDB genutzt um compilierte Dateien zu Untersuchen und Informationen zu erlangen. Später wird durch einen Buffer-Overflow eine Funktion in einem Programm ausgeführt, die nicht hätte ausgeführt werden sollen.

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Glossar 1

2 Aufbau **2**

3 Durchführung **3**

3.1 Assembly 3

3.2 GDB 4

3.3 Buffer Overflow Exploit 8

4 Responsible disclosure **11**

5 Zusammenfassung und Fazit **12**

6 LLM **13**

1 Einleitung

In Programmiersprachen wie z.B. C haben Arrays eine feste Größe und können so nachträglich nicht länger oder kürzer gemacht werden. Die kann jedoch zu Problemen führen. Ein Array reserviert einen bestimmten Speicher, jedoch werden in der gleichen Speichereinheit auch andere Daten gespeichert.

Diese Speichereinheit wird auch *Stack* gennant. Auf diesem Stack liegen verschiedene Frames. Ein Frame besteht dabei unter anderem aus den lokalen Variablen, dem *Saved Frame Pointer* (SFP) und der *Return Adresse*.

Wenn nun eine Funktion in einem Programm aufgerufen wird, dann ist die *Return Adresse* die Adresse, zu welcher zurück gesprungen werden soll. Wenn nun ein übergebenes Argument in ein lokales Array geschrieben werden soll, kann es passieren, dass das Argument länger ist als der reservierte Speicher. In diesem Fall würde der *SFP* und kurz darauf auch die *Return Adresse* überschrieben werden. Da die *Return Adresse* nun überschrieben werden kann, kann eine spezifische Adresse dort eingefügt werden. So können funktionen ausgeführt werden, die nicht erreichbar sein sollten.

Solch einen Buffer-Overflow kann an Angreifer nutzen, um Schadcode auf einem Computer auszuführen und möglicherweise eine Shell mit Administratorrechten zu erhalten. Um diesen Angriff besser zu verstehen, wird in diesem Versuch ein solcher Angriff Schritt für Schritt durchgeführt und erklärt.

1.1 Glossar

Shell Schnittstelle zwischen Copmuter-Kernel und Benutzer

Buffer-Overflow Überlauf eines reservierten Speichers

RBP Speichert die Adresse des aktuellen Stack-Frames

SFP Saved Frame Poiner

RAX Speichert den Rückgabewer einer Funktion

EAX Niederwertige 32 Bit von RAX

2 Aufbau

Die folgenden Versuche wurden im Netzlabor ID 2/168 unternommen.
Wir führen den Versuch zu zweit als Gruppe 43 am PC mit der IP-Adresse 192.168.1.42 durch.
Das Betriebsystem ist ein Ubuntu 22.04 LTS.

Es wurden verschiedene Programme und Befehle genutz. Diese sind:

Software	**Version**
GNU Nano	6.2
GNU gdb	12.0.90
Python	3.10.4
cat	GNU coreutils 8.32

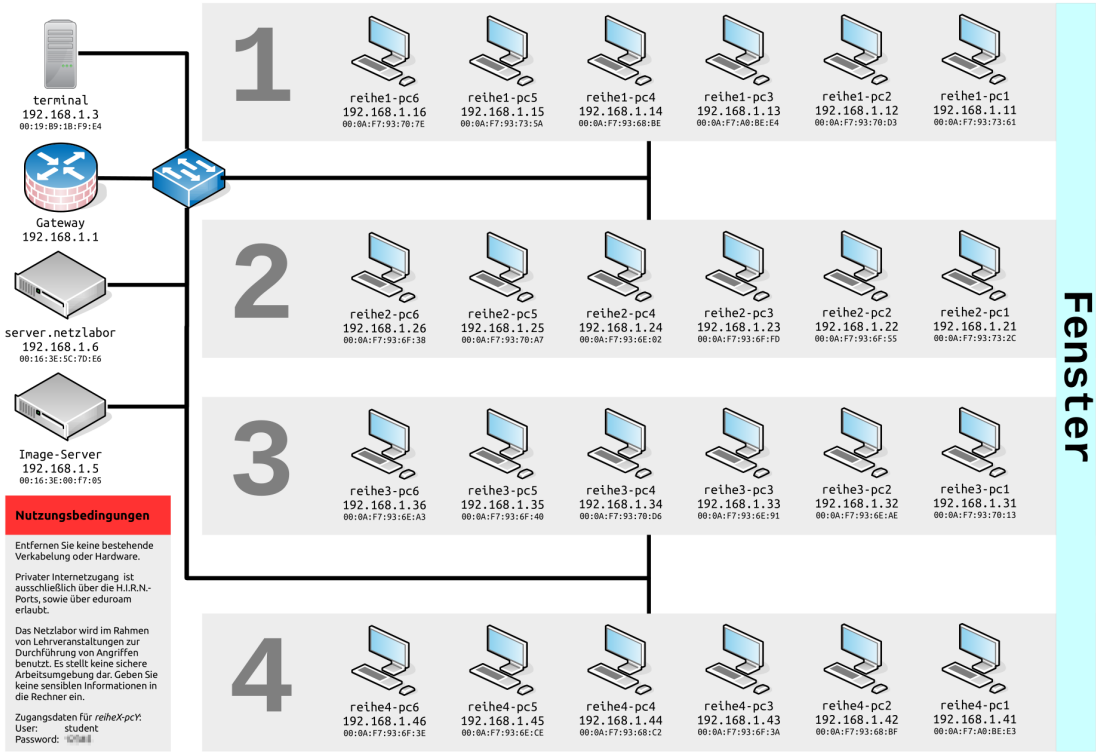

Abbildung 1: Netzplan Raum ID 2/168, Stand 01.05.2017

Die Aufgabenstellung [2] wurde bereutgestellt übe die Veranstaltungsseite [1]

3 Durchführung

Im Folgenden werden die einzelnen Aufgaben durchgeführt, erläutert und das Ergebnis präsentiert.

3.1 Assembly

Damit ein Computer ein Programm, das von einem Menschen geschrieben wurde, verstehen kann muss dieses konvertiert werden. Eine der grundlegendsten und direktesten Weisen damit Mensch und Maschine kommunizieren können ist die Sprache Assembler. Diese besteht aus Anweisungen die der Computer direkt in Bits und Bytes, sogenannten Opcode, übersetzten und dadurch ausführen kann. Da ein Mensch sich leichter Buchstaben als Reihenfolgen von 0 und 1 merken kann gibt es Mnemonics, welche als leicht merkbare Kürzel agieren.

Damit Daten gespeichert werden können, stehen dem Prozessor sogenannte Register zur Verfügung. Ein solches Register ist je nach Prozessorarchitektur unterschiedlich. Der in diesem Versuch genutzte Computer hat eine Registergröße von 64 Bit, also 8 Byte. Register können über ihre Adresse angesprochen werden, einigen Registern wurde jedoch ein Name zugeteilt. Die Bezeichnung *eax* bezieht sich auf die niedrigsten 32 Bit des *RAX* Registers.

Es wird der folgende Auszug eines Programms in Assembler betrachtet.

```
1 mov     eax, 0x10
2 cmp     eax, 10
3 jle     0xdeadbeef
4 jmp     0xcafebabe
```

Die erste Zeile nutzt die Mnemonic *mov* um den Hexadezimalwert 0x10, 16 in dezimal, in das Register *eax* zu bewegen. In der nächsten Zeile wird durch die Mnemonic *cmp* der Wert des Registers *eax* mit dem Dezimalwert 10 vergleichen. Die Mnemonic subtrahiert den zweiten Wert vom ersten Wert und updated bestimmte Flags im Register, um Informationen über das Ergebnis zu speichern. So kann die *jle* Mnemonic wissen, ob der erste Wert kleiner oder gleich dem zweiten Wert ist. Ist dies der Fall, springt das Programm zu der Adresse *0xdeadbeef*, ansonsten passiert nichts und die nächste Zeile wird ausgeführt. Diese Springt über die *jmp* Mnemonic zu der Adresse *0xcafebabe*.

Da das Register *eax* den Dezimalwert 16 enthält und 16 nicht kleiner oder gleich 10 ist, wird kein Sprung in Zeile 3 ausgeführt sondern erst in Zeile 4. Es wird also zu der Adresse *0xcafebabe* gesprungen.

Damit verschiedene Programme von verschiedenen Programmieren gleich verstanden, genutzt und geändert werden können gibt es einige Konventionen an welche jeder Programmieren sich halten sollte, es ist allerdings keine Pflicht. Es sollten einige Register immer für die gleichen Aktionen genutzt werden. So sollte das *RAX* Register dazu benutzt werden, am Ende den Rückgabewert zu speichern. Das Register kann jedoch innerhalb der Funktion für alles genutzt werden.

Das folgende Assembler Programm berechnet die Funktion $f(x) = 2x + 3 * (x + 5)$ für $x = 2$.

```
xor     rdi, rdi    ; Register nullen
xor     rbx, rbx    ; Register nullen
xor     rax, rax    ; Register nullen
mov     rdi, 2      ; x=2
mov     rbx, rdi    ; rbx = rdi = 2
add     rbx, 5      ; rbx = rdi+5 = 7
mov     rax, rbx    ; rax = 7
add     rax, rbx    ; rax = 2*7
add     rax, rbx    ; rax = 3*7
add     rdi, rdi    ; rdi = 2*2
add     rax, rdi    ; rax = 21+rdi = 21+4 = 25
```

3.2 GDB

Wenn ein Programmierer ein Programm schreibt werden oft Variablen genutzt um Daten in diesen für eine gewisse Zeit zu speichern. In Assembler werden diese Daten in Registern gespeichert. Zu wissen in welchen Registern die Daten stehen kann hilfreich sein, um ein Programm im laufenden Betrieb zu verändern. Ein Tool um ein Programm Schritt für Schritt auszuführen und den Assembler Code zu sehen ist *gdb*.

Im folgenden soll das Programm *vals* untersucht werden. Das Ziel ist es, durch statische und dynamische Analyse herauszufinden, an welcher Adresse die Variablen 'val1' und 'val2' initialisiert werden. Im Folgenden kann der Sourcecode des Programm gesehen werden.

```
1  #include <stdio.h>
2  #include <stdint.h>
3
4  int main(void) {
5      uint64_t values[] =
       {123,1231,7583,126786,7879123,17849923,27349,123891023,3742,1238912,12830123};
6
7      uint64_t val1= 0;
8
9      for(int i = 0; i < (sizeof(values)/sizeof(*values)); i++) {
10         val1 += values[i];
11     }
12
13     uint64_t val2 = val1;
14
15     for(int i = (sizeof(values)/sizeof(*values)); i > 0; i--) {
16         if(i %2 == 0) {
17             val2 -= values[i];
18         }
19         else {
20             val2 += values[i];
21         }
22     }
23
24     printf("damn I don't know what my values are, can you help me?\n");
25
26     return 0;
27 }
```

Durch die statische Analyse des Sourcecodes kann erkannt werden, dass die Variable 'var1' in Zeile 7 als *unsigned integer* mit einer Länge von 64 Bit und dem Wert 0 initialisiert wird. Zudem geschiet die Zuweisung unmittelbar nachdem ein Array mit verschiedenen Werten initialisiert wurde. Mit der Hilfe dieser Informationen kann die Adresse von 'val1' in der dynamischen Analyse gefunden werden. Dazu wird das kompilierte Programm *vals* mit *gdb* ausgeführt. Weiterhin ist bekannt, dass die Funktion die untersucht werden soll den Namen *main* hat. So kann diese Funktion per *gdb* in Assemblercode disassembled werden. Dies geschieht über den Befehl *pdisas* gefolgt von dem Namen der Funktion.

```
student@Reihe4-PC2:~/Praktikum/Exploiting/exercises/gdb/1$ gdb vals
[...]
gdp-peda$ pdisas main
```

Die Ausgabe ist eine Reihe von Assembler-Anweisungen (siehe Abb. 2). Zu Beginn gibt es eine Passage, in der viele *mov* Mnemonics genutzt werden um Hexadezimalwerte in Registern zu speichern. Wenn die hexadezimalen Zahlen in dezimal umgerechnet werden, kann erkannt werden, dass es sich um die Zahlen des Arrays hält. Dabei entspricht *0xc3c5ab* der Zahl *12830123*, also der letzten Zahl im Array. Aus der statischen Analyse geht hervor, dass die nächste Variable die Initialisiert wird 'val1' ist. Im Assemblercode gibt es nach der letzten *mov* Mnemonic für das Array eine weitere (siehe markiert in Abb. 2), welche den Wert *0x0* in das Register mit der Adresse *rbp-0x70* schreibt. Die Adresse an der 'val1' initiallisiert wird ist somit *0x00000000004011c9*.

```
student@Reihe4-PC2: ~/Praktikum/Exploiting/exercises/gdb/1
0x0000000000401171 <+27>:    mov    QWORD PTR [rbp-0x60],0x7b
0x0000000000401179 <+35>:    mov    QWORD PTR [rbp-0x58],0x4cf
0x0000000000401181 <+43>:    mov    QWORD PTR [rbp-0x50],0x1d9f
0x0000000000401189 <+51>:    mov    QWORD PTR [rbp-0x48],0x1ef42
0x0000000000401191 <+59>:    mov    QWORD PTR [rbp-0x40],0x7839d3
0x0000000000401199 <+67>:    mov    QWORD PTR [rbp-0x38],0x1105e43
0x00000000004011a1 <+75>:    mov    QWORD PTR [rbp-0x30],0x6ad5
0x00000000004011a9 <+83>:    mov    QWORD PTR [rbp-0x28],0x7626d4f
0x00000000004011b1 <+91>:    mov    QWORD PTR [rbp-0x20],0xe9e
0x00000000004011b9 <+99>:    mov    QWORD PTR [rbp-0x18],0x12e780
0x00000000004011c1 <+107>:   mov    QWORD PTR [rbp-0x10],0xc3c5ab
0x00000000004011c9 <+115>:   mov    QWORD PTR [rbp-0x70],0x0
0x00000000004011d1 <+123>:   mov    DWORD PTR [rbp-0x78],0x0
0x00000000004011d8 <+130>:   jmp    0x4011ec <main+150>
0x00000000004011da <+132>:   mov    eax,DWORD PTR [rbp-0x78]
0x00000000004011dd <+135>:   cdqe
0x00000000004011df <+137>:   mov    rax,QWORD PTR [rbp+rax*8-0x60]
0x00000000004011e4 <+142>:   add    QWORD PTR [rbp-0x70],rax
0x00000000004011e8 <+146>:   add    DWORD PTR [rbp-0x78],0x1
0x00000000004011ec <+150>:   mov    eax,DWORD PTR [rbp-0x78]
0x00000000004011ef <+153>:   cmp    eax,0xa
0x00000000004011f2 <+156>:   jbe    0x4011da <main+132>
0x00000000004011f4 <+158>:   mov    rax,QWORD PTR [rbp-0x70]
0x00000000004011f8 <+162>:   mov    QWORD PTR [rbp-0x68],rax
```

Abbildung 2: GDB Ausgabe von Assemblercode zu val1

Um die Adresse der Variable 'val2' zu finden, kann zuerst die for-Schleife im Assemblercode gesucht werden (siehe Abb. 3). Zuerst wird eine neue Variable 'i' mit 0 initialisiert. Dies geschieht direkt nachdem 'val1' initialisiert wurde. Dann gibt es einen Sprung zu *main+150* an dieser Stelle wird der Wert von 'i' in das Register *eax* geschrieben. Direkt darauf wird der Wert von *eax* mit dem Wert *0xa* verglichen. Dieser Wert ist *10* in dezimal und wird als Abbruchbedingung der for-Schleife genutzt. Da es 11 Elemente im Array gibt und der Sprung bei größer oder gleich geschiet, wird die Schleife bei einer Abbruchbedingung von 10 und dem Startwert 0 11 Mal ausgeführt. Sobald der Sprung nicht mehr ausgeführt wird, wird das Ergebnis der Berechnungen in das Register *rax* geschrieben und dann in das Register mit der Adresse *rbp-0x68* (siehe markiert in Abb. 3). In dem Sourcecode kann gesehen werden, dass nach der for-Schleife die Variable 'val2' initialisiert wird. Daraus kann geschlossen werden, dass die Adresse *rbp-0x68* die Adresse der Variable 'val2' ist und die Variable an der Adresse *0x00000000004011f8* initiallisiert wird.

```
student@Reihe4-PC2: ~/Praktikum/Exploiting/exercises/gdb/1
0x00000000004011d1 <+123>:   mov    DWORD PTR [rbp-0x78],0x0
0x00000000004011d8 <+130>:   jmp    0x4011ec <main+150>
0x00000000004011da <+132>:   mov    eax,DWORD PTR [rbp-0x78]
0x00000000004011dd <+135>:   cdqe
0x00000000004011df <+137>:   mov    rax,QWORD PTR [rbp+rax*8-0x60]
0x00000000004011e4 <+142>:   add    QWORD PTR [rbp-0x70],rax
0x00000000004011e8 <+146>:   add    DWORD PTR [rbp-0x78],0x1
0x00000000004011ec <+150>:   mov    eax,DWORD PTR [rbp-0x78]
0x00000000004011ef <+153>:   cmp    eax,0xa
0x00000000004011f2 <+156>:   jbe    0x4011da <main+132>
0x00000000004011f4 <+158>:   mov    rax,QWORD PTR [rbp-0x70]
0x00000000004011f8 <+162>:   mov    QWORD PTR [rbp-0x68],rax
0x00000000004011fc <+166>:   mov    DWORD PTR [rbp-0x74],0xb
0x0000000000401203 <+173>:   jmp    0x401231 <main+219>
0x0000000000401205 <+175>:   mov    eax,DWORD PTR [rbp-0x74]
0x0000000000401208 <+178>:   and    eax,0x1
0x000000000040120b <+181>:   test   eax,eax
0x000000000040120d <+183>:   jne    0x40121f <main+201>
0x000000000040120f <+185>:   mov    eax,DWORD PTR [rbp-0x74]
0x0000000000401212 <+188>:   cdqe
0x0000000000401214 <+190>:   mov    rax,QWORD PTR [rbp+rax*8-0x60]
0x0000000000401219 <+195>:   sub    QWORD PTR [rbp-0x68],rax
0x000000000040121d <+199>:   jmp    0x40122d <main+215>
0x000000000040121f <+201>:   mov    eax,DWORD PTR [rbp-0x74]
```

Abbildung 3: GDB Ausgabe von Assemblercode zu val2

Ein weiteres Programm das untersucht werden soll ist *crypto*. Dieses Programm nimmt einen String entgegen und verschlüsselt diesen. Der Sourcecode ist im folgenden zu sehen.

```
#include <stdio.h>
#include <stdlib.h>

void fatal(void) {
    printf("error.\n");
    exit(1);
}

int main(int c, char **v) {

    if(c != 2)
        fatal();

    printf("nom nom, I'll digest your string...\n");


    for(int i = 0; v[1][i]; i++) {
        printf("%c",v[1][i] + 10);
    }

    printf("\nhere you go!\n");

    return 0;
}
```

Wenn das Programm ausgeführt wird, darf nur ein Argument übergeben werden. Dieses Argument ist der String der verschlüsselt werden soll. In der Variable 'c' wird die Anzahl der Argumente gespeichert. Da der Aufruf des Programms auch als Argument gilt, wird zuerst überprüft, ob die Anzahl der Argumente genau 2 ist. Wenn 'c' einen anderen Wert hat, wird das Programm abgebrochen. Der Hauptteil des Programm ist die for-Schleife. In dieser wird der übergebene String Zeichen für Zeichen gelesen. Dann wird jedes Zeichen einzeln verschlüsselt. Die Verschlüsselung besteht hierbei aus dem addieren von 10 auf den ASCII-Wert des Zeichens. Es ist also eine Caesar-Chiffre. Nachdem verändern wird das Zeichen ausgegeben. Wenn das Programm beendet ist, kann auf dem Terminal der veränderte String gelsen werden. Die Verschlüsselung ist nicht sicher, da zum einen der Schlüssel fest bei 10 im Programmcode steht und im weiteren auf diese Art und Weise nicht genug verschiedene Schlüssel existieren. Dies macht einen Brute-Force Angriff sehr einfach. Um die Adresse zu finden an welcher die Zeichen verändert werden kann wie zuvor *gdb* genutzt werden. Nachdem das Programm *crypto* mit *gdb* gestartet und disassembled wurde, kann nach einer *add* Mnemonic, welche *10* oder *0xa* zu etwas addiert, gesucht werden. Wie in Abbildung 4 zu sehen gibt es lediglich eine *add* Mnemonic bei der *0xa* auf ein Register addiert wird. Diese Anweisung geschiet an der Adresse *0x00000000004011e6* wie an der markierten stelle in Abbildung 4 zu sehen ist.

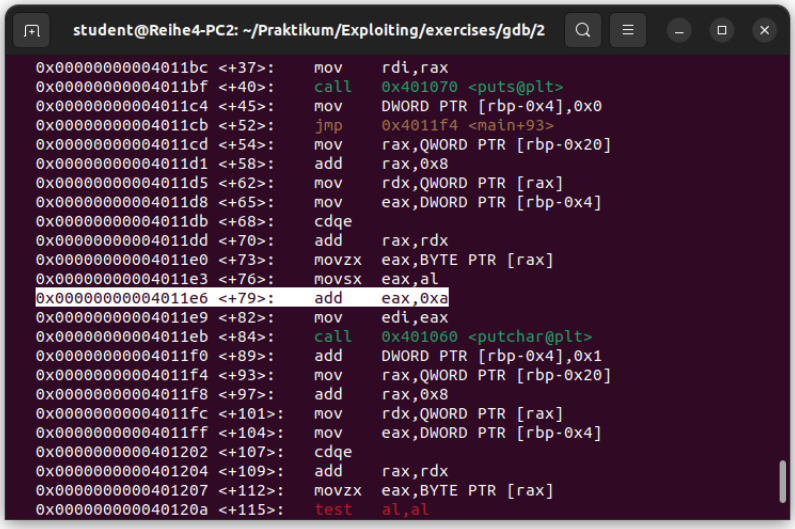

Abbildung 4: GDB Ausgabe von Assemblercode zur Änderung der Zeichen

3.3 Buffer Overflow Exploit

Um die bisher gesammelten Erkenntnisse anzuwenden soll ein Buffer-Overflow Angriff durchgeführt werden. Dazu wird die Sourcecode Datei 'buf.c' untersucht.

```
#include<stdio.h>
#include<stdlib.h>

int func() {
    char lololol[1024];
    gets(lololol);

    return 1;
}

int foo() {
    printf("haxxo00r\n");
    exit(1337);
}

int main(void) {
    printf("enter some chars plzz\n");
    func();
    printf("no win :'(\n");
}
```

Der erste Schritt für einen Buffer-Overflow Angriff ist, eine Möglichkeit zu finden wie ein Angreifer direkten Einfluss auf den Stack nehmen kann. In dem Sourcecode kann gesehen werden, dass die Funktion *func* Speicher reserviert um dort einen Array aus chars, also Zeichen, abzulegen. Der Array biete speicher für 1024 Zeichen. Mit der Funktion *gets* wird die Eingabe eines Benutzters gelsen. Dies ist also die Chance für einen Angreifer einen Buffer-Overflow zu erzeugen.

Um zu zeigen, dass ein Angrif funktioniert, soll die versteckte Funktion *foo* ausgeführt werden. Diese wird nie durch das Programm aufgerufen und dient so als Beweis, dass der Angriff erfolgreich ausgeführt wurde. Die letzte Ausgabe des Programms sollte immer 'no win :(́' sein, wenn jedoch der Angriff erfolgreich war, ist die letzte Ausgabe 'haxxo00r'. Für den Angriff muss die Adresse der auszuführenden Funktion bekannt sein. Diese kann mit Hilfe von *gdb* ermittelt werden (siehe markiert in Abb. 5). Über den Befehl *info functions* werden alle Funktionen mit der jeweiligen Adresse ausgegeben. Die Adresse der Funktion *foo* ist demnach *0x0000000000400600*.

```
student@Reihe4-PC2:~/Praktikum/Exploiting/exercises/gdb/1$ gdb vals
[...]
gdp-peda$ info functions
```

Der Buffer-Overflow geschieht durch den Array in der Funktion *func*. Um den Angriff ausführen zu können, muss daher die Speicherstelle des Array ermittelt werden. Auch dies ist mittels *gdb* möglich. Dazu wird ein Breakpoint in der Funktion *func* gesetzt und anschließend das Programm ausgeführt. Aus dem Sourcecode ist bekannt, dass der Array Platz für 1024 Zeichen bietet. Die Zahl 1024 ist in hexadezimaler Schreibweise *0x400*.

```
gdp-peda$ b func
Breakpoint 1 at 0x4005da
gdp-peda$ r
```

In der Ausgabe ist nun ein Ausschnitt des Assemblercodes zu sehen (siehe Abb. 6). Da der ermittelte Anfang der Funktion bei *0x4005da* liegt kann erkannt werden, das die Größe des Array *0x400* beträgt, da diese Zahl mit der *sub* Mnemonic von rsp abgezogen wird. Die Speicherstelle des Array liegt laut Ausgabe bei *rbp-0x400*. Dies ergibt Sinn, da auf dem *Stack Frame* die lokalen Variablen gespeichert werden und in dem Register *rbp* die Adresse des aktuellen *Stack Frame* gespeichert wird. Um die versteckte Funktion bei der Rückkehr aus der Funktion aufzurufen, muss die Rückkehr-Adresse mit der Adresse der verstecken Funktion *foo* überschrieben werden. Dazu muss

```
student@Reihe4-PC2: ~/Praktikum/Exploiting/exercises/exp...
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./vuln...
(No debugging symbols found in ./vuln)
gdb-peda$ info functions
All defined functions:

Non-debugging symbols:
0x0000000000400498  _init
0x00000000004004c0  puts@plt
0x00000000004004d0  gets@plt
0x00000000004004e0  exit@plt
0x00000000004004f0  _start
0x0000000000400520  deregister_tm_clones
0x0000000000400550  register_tm_clones
0x00000000004005a0  __do_global_dtors_aux
0x00000000004005d0  frame_dummy
0x00000000004005da  func
0x0000000000400600  foo
0x000000000040061a  main
0x0000000000400650  __libc_csu_init
0x00000000004006c0  __libc_csu_fini
0x00000000004006c4  _fini
gdb-peda$
```

Abbildung 5: GDB Ausgabe der Funktionen

```
student@Reihe4-PC2: ~/Praktikum/Exploiting/exercises/exp...
R8 : 0x0
R9 : 0x6012a0 ("enter some chars plzz\n")
R10: 0x77 ('w')
R11: 0x246
R12: 0x7fffffffe0e8 --> 0x7fffffffe3ee ("/home/student/Praktikum/Exploiting/exer
cises/exploiting/vuln")
R13: 0x40061a (<main>:  push   rbp)
R14: 0x0
R15: 0x7ffff7ffd040 --> 0x7ffff7ffe2e0 --> 0x0
EFLAGS: 0x206 (carry PARITY adjust zero sign trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x4005d5 <frame_dummy+5>:    jmp    0x400550 <register_tm_clones>
   0x4005da <func>:     push   rbp
   0x4005db <func+1>:   mov    rbp,rsp
=> 0x4005de <func+4>:   sub    rsp,0x400
   0x4005e5 <func+11>:  lea    rax,[rbp-0x400]
   0x4005ec <func+18>:  mov    rdi,rax
   0x4005ef <func+21>:  mov    eax,0x0
   0x4005f4 <func+26>:  call   0x4004d0 <gets@plt>
[------------------------------------stack-------------------------------------]
0000| 0x7fffffffdfc0 --> 0x7fffffffdfd0 --> 0x1
0008| 0x7fffffffdfc8 --> 0x400634 (<main+26>:   lea    rdi,[rip+0xb8]        # 0
x4006f3)
0016| 0x7fffffffdfd0 --> 0x1
```

Abbildung 6: GDB Ausgabe nach dem Breakpoint

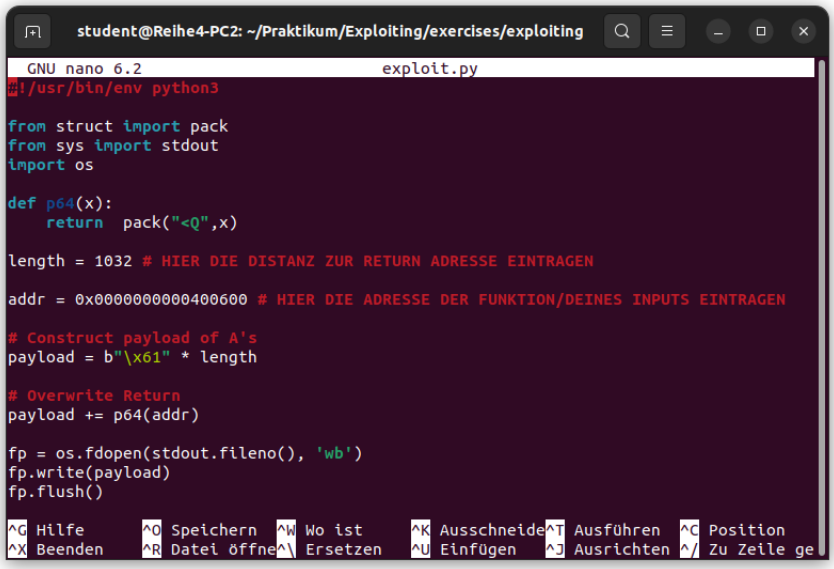

Abbildung 7: Python-Script als exploit

die Distanz zwischen dem Beginn des Inputs bis zum *Frame Pointer* (RBP) berechnet werden. Da die einzige lokale Variable das Array ist, wird die Distanz kaum größer als das Array selbst sein. Das einzige was noch zwischen der Rückkehr-Adresse und dem Array liegt ist der *Saved Frame Pointer*. Da der SFP lediglich ein Register benutzt und ein Register 8 Byte groß ist, sollte die Distanz zwischen Arraybeginn und RBP bei $1024 + 8 = 1032$ liegen.

Da nun alle notwendigen Informationen für den Angriff bekannt sind, kann das Python-Script *exploit.py* bearbeitet und dann ausgeführt werden (siehe Abb. 7). Das Script schreibt die eingetragene Anzahl an A's in den Standardinput. Das Programm *vuln* wird diesen Lesen, was dann zum Buffer Overflow führt.

Um den Buffer Overflow herbeizuführen wird das Python-Script gestartet und die Ausgabe per piping in die Standardeingabe des anzugreifenden Programms geschrieben.

```
student@Reihe4-PC2:~/Praktikum/Exploiting/exercises/exploiting$ ./exploit.py | ./vuln
enter some chars plzz
haxxo00r
```

An der Ausgabe kann gesehen werden, dass der Angriff erfolgreich war. Wie zu Beginn angenommen ist die Ausgabe 'haxxo00r', was bestätigt, dass die Funktion *foo* erfolgreich ausgeführt wurde.

4 Responsible disclosure

Sehr geehrte Damen und Herren,

bei einer kürzlich erfolgten Überprüfung Ihrer neuen Software zur Verbreitung medizinischer Daten sind erheblicher Sicherheitslücken gefunden worden. Angreifern ist es möglich, die Patienteneingaben auszunutzen um einen Buffer-Overflow Angriff durchzuführen.

Bei einem Buffer-Overflow Angriff werden mehr Daten in ein Feld eingegeben, als das Programm erwartet hat. Dies kann dazu führen, dass unerwünschte Programme und Schadcode ausgeführt werden. Weiterhin kann eine Shell, eine Mensch Computer Schnittstelle, geschaffen werden, welchen Angreifern die volle Kontrolle über den jeweiligen Computer ermöglichen kann. Da so der Computer und dadurch auch das Netzwerk durch verschiedenste Schadsofware infiziert werden kann, sind die möglichen Schäden sehr hoch und die Lücke als ernstzunehmend einzustufen.

Um diese Art Angriffe zu verhindern, kann eine maximale Länge an Nutzereingaben festgelegt werden. Hierbei ist zu beachten, dass die Länge passend gewählt wird um genug Informationen zu erhalten, den Copmuter aber nicht mit zu viel Daten den Speicher unnltig füllen zu lassen.

Für genauere Informationen und Absprachen suchen wir den Kontakt zu Ihren Softwareentwickler um die notwendigen Änderungen zu besprechen. Auf der hohen Schäden die entstehen können, sollte die Lücken in 30 Werktagen geschlossen werden.

Mit freundlichen Grüßen,

Nico Steppeler und Sabri Oztürk, Berater für IT-Sicherheit

5 Zusammenfassung und Fazit

In diesem Versuch haben wir uns mit Buffer-Overflow Angriffen und Assembler beschäftigt. Durch Programme wie *gdb* ist es möglich, bereits Kompilierte Programme Schritt für Schritt auszuführen und die einzelnen Register anzuschauen und zu verändern. Die Informationen die aus Registern gewonnen werden können sind wichtig um Angriffe vorzubereiten und erfolgreich durchzuführen.

Durch die Beeinflussung durch *gdb* ist es möglich, Funktionen und Code auszuführen, der nicht ausgeführt werden sollte. Wir konnten so die Ausgabe eines Programms durch das ausführen einer versteckten Funktion ändern. Es kann aber auch andere Code ausgeführt werden. Es ist beispielsweise möglich eine Shell per Buffer-Overflow zu spawnen und so direkt Befehle an den Computer zu senden.

Dieser Versuch hat uns sehr gut gefallen. Die Aufgaben waren interessant und auch die Anwendungen bieten weitere spannende Themen. Die Erklärungen zu den verschiedenen Themen waren gut und verständlich. Vor allem was ein Stack ist und wie dieser funktioniert ist interessant zu wissen. Die Verwendung von *gdb* hat uns dazu motiviert, weitere Funktionalitäten und Möglichkeiten zu entdecken. Alles in allem ein sehr gelungener Versuch.

6 LLM

In diesem Versuch wurden keine Large-Language Models(LLM) verwendet. Die Texte sind alleine die Arbeit der Mitglieder der Gruppe 43 wie zubeginn angegeben.

Literatur

[1] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Bachelor Grundlagenpraktikum ITS: Veranstaltungsseite der Fakultät für Elektrotechnik und Informationstechnik. URL `https://informatik.rub.de/studium/lehrveranstaltungen/`.

[2] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Aufgabenstellung zum Praktikumsversuch. Bochum, 2023.